



UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO

Nombre de la Organización:

UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO (UTEQ)

Presenta:

VICTOR GUADALUPE LOPEZ CEBREROS

Grupo:

IDGS17

Unidad 2. Práctica 1. Instalación, Configuración y ejecución de SonarQube

1. Objetivo

Configurar SonarQube Community Edition dentro de un contenedor Docker e integrarlo con un proyecto ASP.NET Core 10, con el fin de detectar de manera automatizada vulnerabilidades de seguridad, defectos funcionales, problemas de mantenibilidad y niveles de cobertura de pruebas, incorporando el análisis estático de código como parte del ciclo de desarrollo.

De manera específica, la práctica persigue instalar y ejecutar SonarQube junto con su base de datos PostgreSQL mediante Docker Compose, asegurar la cuenta administrativa y crear un proyecto local, configurar SonarScanner para .NET y vincularlo al servidor mediante un token de análisis, ejecutar un análisis estático completo e interpretar sus resultados, y finalmente corregir al menos tres hallazgos validando la mejora con un segundo análisis.

2. Marco conceptual

El análisis estático de código, conocido como SAST (Static Application Security Testing), consiste en examinar el código fuente sin ejecutarlo, buscando patrones asociados a defectos y vulnerabilidades conocidas. A diferencia de las pruebas dinámicas, que requieren la aplicación en funcionamiento y solo alcanzan las rutas efectivamente ejecutadas, el análisis estático puede recorrer la totalidad del código, incluidas las rutas de error y los casos límite que rara vez se prueban de forma manual.

SonarQube implementa este enfoque mediante reglas que clasifican los hallazgos en cuatro categorías: bugs, vulnerabilidades, security hotspots y code smells. Adicionalmente incorpora el concepto de Quality Gate, un conjunto de condiciones que el código debe satisfacer para considerarse apto para su liberación, lo que permite convertir criterios de calidad y seguridad en una verificación automática y objetiva dentro del pipeline de integración continua.

El valor principal de la herramienta radica en el desplazamiento del control hacia etapas tempranas del desarrollo. Detectar una vulnerabilidad durante la codificación tiene un costo de corrección sustancialmente menor al de detectarla en producción, donde además existe la posibilidad de que ya haya sido explotada.

3. Desarrollo de la práctica

3.1 Definición de la infraestructura con Docker Compose

Se construyó un archivo docker-compose.yml que define dos servicios interdependientes: el servidor de SonarQube, a partir de la imagen sonarqube:community, y su base de datos, a partir de postgres:15. La orquestación mediante Compose permite levantar y destruir el entorno completo con un solo

comando, garantizando que la configuración sea reproducible en cualquier equipo sin depender de instalaciones manuales.

La declaración de volúmenes con nombre resulta especialmente relevante dentro de esta configuración. Sin ellos, la totalidad de los proyectos, hallazgos históricos y ajustes de configuración se perderían al eliminar el contenedor, dado que el sistema de archivos de un contenedor es efímero por diseño. Los volúmenes desacoplan el ciclo de vida de los datos del ciclo de vida del contenedor, permitiendo actualizar la imagen de SonarQube sin perder el historial acumulado. Por su parte, la directiva `depends_on` establece el orden de arranque, evitando que SonarQube intente conectarse a una base de datos que aún no ha terminado de inicializarse.

La conexión entre ambos servicios se resuelve mediante el DNS interno de Docker: la cadena `JDBC` apunta al nombre del servicio `sonar_db` en lugar de a una dirección IP, la cual sería asignada dinámicamente y cambiaría en cada arranque.

3.2 Ajuste de parámetros del sistema operativo

SonarQube utiliza Elasticsearch de forma interna para indexar el código analizado, y Elasticsearch exige que el parámetro `vm.max_map_count` del kernel tenga un valor mínimo de 524288. Dicho parámetro define el número máximo de áreas de memoria que un proceso puede mapear; cuando resulta insuficiente, el contenedor arranca y se detiene de inmediato, registrando un error de bootstrap en los logs sin que la interfaz llegue a estar disponible en ningún momento.

De manera complementaria se ajustó `fs.file-max`, que determina el número máximo de descriptores de archivo que el sistema puede mantener abiertos simultáneamente. En instalaciones de Docker Desktop sobre Windows ambos valores se aplican de forma automática, por lo que el ajuste manual solo resulta necesario al ejecutar Docker directamente dentro de WSL2 o sobre un anfitrión Linux.

3.3 Arranque y verificación de los contenedores

Los contenedores se levantaron en segundo plano y se verificó que ambos permanecieran en estado *running*. Posteriormente se monitorearon los logs del servicio hasta encontrar el mensaje que confirma la disponibilidad del sistema.

Esta verificación mediante logs no es un paso opcional. El hecho de que un contenedor aparezca como *running* únicamente indica que su proceso principal continúa vivo, no que la aplicación esté lista para atender peticiones. SonarQube requiere un tiempo considerable de inicialización, durante el cual crea el esquema de base de datos en el primer arranque y levanta el motor de indexación. Intentar acceder a la interfaz antes de ese punto produce errores de conexión que suelen interpretarse erróneamente como fallas de configuración.

3.4 Acceso a la interfaz y aseguramiento de la cuenta administrativa

Se accedió a la consola web en el puerto 9000 utilizando las credenciales por defecto y, siguiendo el requerimiento del propio sistema, se sustituyó de inmediato la contraseña por una de mayor longitud y complejidad.

Este paso constituye en sí mismo una lección de seguridad pertinente al contenido de la materia. Las credenciales por defecto son públicas y están documentadas para toda instalación de SonarQube, por lo que un servicio expuesto que las conserve equivale funcionalmente a un servicio sin autenticación. Existen herramientas de escaneo masivo que prueban combinaciones conocidas de forma automatizada sobre rangos completos de direcciones, de modo que el tiempo de exposición efectivo de un servicio así no se mide en semanas sino en minutos. El propio SonarQube reconoce este riesgo al forzar el cambio de contraseña en el primer inicio de sesión, en lugar de limitarse a recomendarlo.

3.5 Creación del proyecto y generación del token de análisis

Se creó un proyecto local y se generó un token del tipo Project Analysis Token, cuyo alcance queda limitado exclusivamente a ese proyecto.

La elección del tipo de token no constituye un detalle menor. Un User Token hereda la totalidad de los permisos de la cuenta que lo emite, de manera que su filtración comprometería todos los proyectos accesibles para ese usuario, incluida la posibilidad de alterar configuraciones globales. Un Project Analysis Token, en cambio, únicamente autoriza el envío de resultados de análisis de un proyecto específico. Aplicar el principio de privilegio mínimo en este punto reduce de forma directa el impacto potencial de una filtración, sin costo alguno en funcionalidad.

Resulta igualmente significativo que el token se muestre una sola vez y que el servidor almacene únicamente su hash. Esta decisión de diseño implica que ni siquiera un administrador puede recuperar el valor de un token existente, lo cual constituye la misma práctica que se aplica al almacenamiento de contraseñas.

3.6 Instalación de SonarScanner y manejo del secreto

SonarScanner para .NET se instaló como herramienta global del SDK. El token se almacenó posteriormente como variable de entorno, y no dentro de comandos, scripts o archivos de configuración del proyecto.

La razón de esta separación es que appsettings.json, los scripts de compilación y cualquier archivo versionado terminan formando parte del historial de Git, y ese historial es permanente. Eliminar el archivo en un commit posterior no elimina el secreto: únicamente lo desplaza hacia atrás en la línea de tiempo, donde permanece plenamente recuperable mediante git log o git show. Un secreto que

llegó a existir en un commit debe considerarse comprometido con independencia de lo que ocurra en commits subsecuentes.

Cabe señalar que el uso de variables de entorno constituye la solución adecuada para un entorno de desarrollo local, pero no representa el estándar en producción. En ese contexto el mecanismo apropiado es un gestor de secretos dedicado, que aporta rotación automática, control de acceso granular y registro de auditoría sobre cada consulta al secreto.

3.7 Ejecución del análisis

El análisis de proyectos .NET se compone de tres fases obligatorias que deben ejecutarse en secuencia: begin, la compilación y end.

Este encadenamiento responde a la arquitectura de la herramienta. La fase begin instala los analizadores de Roslyn dentro del proceso de compilación y prepara el directorio de trabajo donde se depositarán los resultados intermedios. Durante la compilación, esos analizadores se ejecutan como parte del propio compilador de C#, aprovechando el árbol sintáctico y el modelo semántico que este ya construye, lo cual permite un análisis considerablemente más preciso que el basado en simple coincidencia de patrones textuales. Finalmente, la fase end consolida los resultados intermedios, los combina con los reportes de cobertura y los transmite al servidor.

De lo anterior se desprende que el análisis debe envolver a la compilación y no puede ejecutarse de forma aislada sobre el código fuente. En este mismo sentido, la compilación debe forzarse a ser no incremental, dado que una compilación incremental omite los proyectos que no han sufrido cambios y, en consecuencia, los analizadores no llegan a ejecutarse sobre ellos. El resultado es un reporte incompleto o vacío que no arroja error alguno, lo cual constituye una de las causas más frecuentes de análisis aparentemente exitosos pero sin hallazgos.

La ejecución de pruebas con recolección de cobertura se incorporó al mismo ciclo, de manera que el servidor recibe en una sola transmisión tanto los hallazgos del análisis estático como las métricas de cobertura, permitiendo su correlación por archivo y por línea.

3.8 Interpretación de los resultados

Concluido el análisis, el dashboard del proyecto presentó los hallazgos organizados en las categorías de bugs, vulnerabilidades, security hotspots y code smells, junto con las métricas de cobertura, duplicación y deuda técnica estimada.

Se examinó en detalle el security hotspot correspondiente a la construcción de una consulta SQL mediante concatenación de cadenas. Para cada hallazgo, SonarQube proporciona la ubicación exacta en archivo y número de línea, una descripción del

riesgo asociado, el nivel de severidad estimado, una recomendación concreta de corrección y la referencia a los catálogos OWASP y CWE correspondientes. Esta última vinculación resulta particularmente valiosa desde el punto de vista formativo, ya que conecta un hallazgo puntual en el código con la taxonomía estandarizada que la industria utiliza para clasificar vulnerabilidades.

El hotspot se marcó con estado *Acknowledged*, acompañado de un comentario que documenta la intención de sustituir la concatenación por consultas parametrizadas. Este flujo de trabajo refleja la práctica profesional: el hallazgo deja de figurar como pendiente de revisión y queda registrado como una decisión consciente del equipo, con trazabilidad sobre quién lo evaluó, cuándo lo hizo y con qué justificación.

3.9 Corrección de hallazgos y validación

Se aplicaron correcciones sobre los hallazgos identificados, atendiendo tres categorías representativas.

En el caso de la **construcción dinámica de consultas SQL**, la causa raíz consiste en que la entrada proporcionada por el usuario se incorpora directamente al texto de la sentencia, de modo que deja de comportarse como dato y pasa a formar parte de la estructura de la consulta. La corrección consistió en sustituir la concatenación por consultas parametrizadas, con lo cual el motor de base de datos recibe la estructura de la sentencia y los valores por canales independientes.

En el caso de la **condición lógica siempre verdadera**, se trata de una expresión booleana cuyo resultado no depende de sus operandos, generalmente producto de un rango mal delimitado o de un operador lógico incorrecto. Su impacto reside en que la rama alternativa nunca llega a ejecutarse, de manera que una validación aparentemente presente en el código resulta inoperante en tiempo de ejecución. La corrección implicó reformular la condición y agregar una prueba unitaria que verifica ambos caminos posibles.

En el caso de la **variable declarada sin uso**, se trata de código residual de una modificación previa. Aunque el compilador no lo señala como error y no afecta el comportamiento del programa, incrementa la carga cognitiva de quien mantiene el código y con frecuencia indica que una operación prevista quedó incompleta, lo cual puede encubrir un defecto real.

Tras aplicar las correcciones se repitió el ciclo completo de análisis con parámetros idénticos, a fin de que la comparación resultara metodológicamente válida, y se verificó en el dashboard la reducción de los hallazgos previamente reportados.

Sobre este punto conviene precisar que el estado del Quality Gate se evalúa de forma predeterminada sobre el código nuevo o modificado, no sobre la totalidad del proyecto. Este criterio, denominado *Clean as You Code*, evita que la deuda técnica

heredada bloquee de manera permanente los despliegues, mientras exige que todo código nuevo cumpla el estándar establecido. En consecuencia, resulta posible que la puerta de calidad aparezca aprobada aun cuando el proyecto conserve hallazgos históricos sin resolver, situación que responde al diseño de la herramienta y no a un error de configuración.

4. Preguntas de reflexión

4.1 ¿Qué diferencia hay entre un Security Hotspot y un Bug en SonarQube?

La diferencia esencial se encuentra en el grado de certeza de la herramienta y, en consecuencia, en el flujo de trabajo que cada categoría desencadena.

Un **bug** es un defecto respecto del cual SonarQube tiene certeza de que el código se comporta de manera distinta a la pretendida por el programador. Corresponde a la dimensión de fiabilidad y no depende del contexto: una condición que siempre resulta verdadera, una referencia nula potencial o un recurso que nunca se libera son incorrectos en cualquier aplicación, sin importar su propósito. El flujo derivado es directo, ya que el hallazgo simplemente debe corregirse.

Un **security hotspot**, en cambio, señala código sensible desde el punto de vista de la seguridad cuya peligrosidad real depende del contexto de uso, y respecto del cual la herramienta no puede pronunciarse por sí sola. Ejemplos característicos son el empleo de un generador de números pseudoaleatorios, la desactivación de la validación de certificados o la construcción dinámica de una consulta. Cada uno de estos elementos puede resultar perfectamente legítimo o constituir una falla grave según dónde y cómo se emplee: un generador pseudoaleatorio es aceptable para seleccionar un color de interfaz e inaceptable para producir un token de sesión, y el analizador no dispone de información suficiente para distinguir ambos escenarios.

Por esa razón el hotspot no exige corrección sino **revisión humana**, y cuenta con un flujo de estados propio —To Review, Acknowledged, Fixed y Safe— que documenta la decisión adoptada. Si durante la revisión se confirma que el código sí resulta explotable, el hotspot se convierte en una *vulnerability*, categoría que ya representa una falla de seguridad confirmada y que se trata con el mismo carácter obligatorio que un bug.

En síntesis: el bug afecta la calificación de fiabilidad y describe qué hace mal el código; el hotspot afecta la calificación de revisión de seguridad y describe qué requiere ser examinado. El primero responde a un problema de corrección funcional, el segundo a una decisión de diseño que necesita criterio humano. Esta separación evita además un problema práctico relevante: si todo código sensible se reportara como vulnerabilidad, el volumen de falsos positivos llevaría al equipo a ignorar el reporte por completo.

4.2 ¿Por qué es inseguro concatenar directamente el input del usuario en una query SQL? Explique el ataque con un ejemplo de payload.

La causa raíz es la mezcla de dos canales que deben permanecer separados: el canal de instrucciones y el canal de datos. Al concatenar, el programa produce una única cadena de texto que el motor de base de datos analiza sintácticamente en su totalidad. Ese motor no dispone de mecanismo alguno para distinguir qué fragmento fue escrito por el desarrollador y cuál provino de un usuario anónimo; interpreta el conjunto como una sola sentencia legítima. Los caracteres que el desarrollador consideraba simples datos —comillas, guiones, punto y coma— son en realidad símbolos con significado sintáctico, y basta con introducirlos para reescribir la lógica de la consulta.

Considérese la siguiente construcción vulnerable:

```
var query = "SELECT * FROM Users WHERE Username = '" + username +  
            "' AND Password = '" + password + '";
```

Con el siguiente payload introducido en el campo de usuario:

```
' OR '1'='1' --
```

La sentencia que efectivamente recibe el motor queda de la siguiente manera:

```
SELECT * FROM Users WHERE Username = " OR '1'='1' --' AND Password = "
```

La comilla inicial cierra de forma prematura el literal de texto; el operador OR introduce una condición que resulta verdadera para todas las filas de la tabla; y el doble guion comenta el resto de la sentencia, anulando por completo la verificación de la contraseña. El resultado es que la consulta devuelve la totalidad de los usuarios y la aplicación, que típicamente evalúa si obtuvo al menos un registro, concede el acceso autenticando al atacante como el primer usuario de la tabla, que con frecuencia es el administrador.

El impacto no se limita a la evasión de la autenticación. Mediante la cláusula UNION SELECT es posible extraer datos de tablas distintas a la consultada, incluidas las tablas de metadatos que revelan la estructura completa del esquema. Con consultas apiladas, según el proveedor y la configuración, pueden ejecutarse sentencias de modificación o borrado. Y a través de técnicas ciegas, basadas en diferencias de respuesta o en retardos temporales inducidos, resulta factible reconstruir el contenido completo de la base de datos incluso cuando la aplicación no muestra ningún resultado al usuario.

La corrección consiste en emplear consultas parametrizadas. Con ellas, el motor recibe primero la estructura de la sentencia, la compila y solo entonces recibe los valores, que ya no pueden alterar un plan de ejecución previamente fijado. El

payload anterior se busca entonces de forma literal, como si fuera un nombre de usuario ordinario, y el resultado es simplemente que no existe coincidencia. En Entity Framework Core esta protección se obtiene por defecto al utilizar LINQ, siempre que se evite FromSqlRaw con cadenas interpoladas manualmente.

Conviene señalar que el escape manual de caracteres no constituye una solución equivalente. Depende de que el desarrollador contemple todas las variantes de codificación y todos los contextos posibles, y falla ante particularidades como los conjuntos de caracteres multibyte o los identificadores numéricos sin comillas, donde no existe delimitador que cerrar. La parametrización elimina la clase completa de vulnerabilidad; el escape únicamente intenta cubrir los casos que se recordaron.

4.3 ¿Qué es el Code Coverage y por qué SonarQube lo considera relevante para la seguridad?

El code coverage mide el porcentaje de líneas y de ramas de decisión del código que llegan a ejecutarse durante la corrida de las pruebas automatizadas. SonarQube distingue entre cobertura de líneas y cobertura de condiciones, siendo esta última la más significativa, ya que una línea que contiene una condición compuesta puede aparecer como cubierta habiéndose ejecutado únicamente uno de sus caminos posibles.

Su relevancia para la seguridad se sostiene en tres argumentos.

En primer lugar, **el código no cubierto es código cuyo comportamiento nadie ha verificado**. Las rutas que suelen quedar sin pruebas son precisamente las de manejo de errores, validación de entradas y verificación de permisos, es decir, aquellas que concentran los controles de seguridad de la aplicación. Un control que jamás se ejecutó en una prueba es, en la práctica, un control cuya efectividad se desconoce; puede haber quedado inutilizado por un cambio posterior sin que nadie lo advierta, dado que su fallo no produce error visible sino ausencia silenciosa de protección.

En segundo lugar, **la cobertura habilita la remediación segura**. Corregir una vulnerabilidad implica modificar código en funcionamiento, y sin pruebas que respalden el comportamiento esperado, el equipo tiende a evitar los cambios por temor a introducir regresiones. Una base de pruebas sólida reduce el costo percibido de corregir y, en consecuencia, incrementa la probabilidad de que las correcciones efectivamente se realicen en lugar de posponerse indefinidamente.

En tercer lugar, **funciona como indicador indirecto de la disciplina de ingeniería del equipo**. Los proyectos con cobertura sistemáticamente baja suelen presentar también revisiones de código superficiales y ausencia de verificaciones

automatizadas, condiciones bajo las cuales los defectos de seguridad sobreviven por más tiempo antes de ser detectados.

Resulta necesario, sin embargo, precisar una limitación importante. La cobertura mide ejecución, no verificación: una prueba que invoca un método sin comprobar ninguna aserción incrementa la métrica sin aportar garantía alguna sobre la corrección del comportamiento. Del mismo modo, una cobertura del cien por ciento no implica ausencia de vulnerabilidades, puesto que las pruebas verifican el comportamiento previsto y no el comportamiento malicioso; ninguna suite de pruebas convencional intenta inyectar payloads. Por ello SonarQube trata la cobertura como una condición necesaria pero no suficiente dentro del Quality Gate, y la evalúa sobre el código nuevo, exigiendo que todo lo que se agregue venga acompañado de sus pruebas correspondientes en lugar de imponer un umbral global sobre el proyecto completo.

4.4 Si un colega sube el token de SonarQube a GitHub por error, ¿qué pasos seguirías para mitigar el riesgo?

El orden de las acciones es determinante, y el error más frecuente consiste en comenzar por limpiar el historial. Mientras el token siga siendo válido, borrarlo del repositorio no aporta protección alguna: el secreto ya salió del perímetro de control y debe considerarse comprometido desde el instante mismo en que se publicó.

Primero, revocar el token de inmediato. Se accede a la sección de seguridad de la cuenta y se elimina el token expuesto. Esta es la única acción que verdaderamente neutraliza el riesgo, y debe ejecutarse antes que cualquier otra. Los bots que rastrean GitHub de forma continua detectan credenciales publicadas en cuestión de segundos, de modo que el tiempo transcurrido constituye el factor crítico de toda la respuesta.

Segundo, generar un token de reemplazo con el alcance mínimo indispensable y registrarlo en un gestor de secretos apropiado, nunca en un archivo del repositorio.

Tercero, revisar la actividad registrada en el servidor durante la ventana de exposición, verificando si se ejecutaron análisis no reconocidos, si se modificaron configuraciones de proyecto o si se alteraron reglas del Quality Gate. Un token de análisis puede utilizarse para inyectar resultados falsos que hagan aparecer como aprobado un código que en realidad no lo está, lo cual convierte al propio control de calidad en el vector del ataque.

Cuarto, depurar el historial de Git mediante herramientas como git filter-repo o BFG Repo-Cleaner, reescribiendo los commits afectados. Esta reescritura invalida las copias locales del resto del equipo, por lo que requiere coordinación previa. Es indispensable comprender, además, que GitHub conserva en caché los objetos de

commits huérfanos y que cualquier bifurcación del repositorio mantiene su propia copia independiente: la limpieza reduce la exposición pero no garantiza la eliminación completa. Este paso es de higiene, no de mitigación.

Quinto, evaluar el alcance real de la exposición. Si el repositorio era público, la credencial debe considerarse comprometida sin margen de duda. Si era privado, se revisa la lista de colaboradores y los registros de acceso, aunque la revocación sigue siendo obligatoria en ambos escenarios.

Sexto, implementar controles preventivos que eviten la recurrencia: hooks de pre-commit con herramientas de detección de secretos, activación del escaneo con protección de push en la plataforma, e incorporación de la verificación como paso obligatorio del pipeline de integración continua.

Séptimo, documentar el incidente sin atribuir culpas individuales. Que una credencial pueda llegar a un commit constituye una falla del proceso, no de una persona. Los entornos que sancionan estos errores producen reportes tardíos, y en seguridad la demora en el reporte resulta considerablemente más costosa que el error original.

El principio general aplicable es que todo secreto expuesto se rota; jamás se intenta ocultar. De este escenario se desprende además un argumento a favor de establecer caducidad en los tokens: una credencial con vigencia limitada acota de manera automática la ventana de riesgo de cualquier filtración que llegue a pasar inadvertida.

4.5 ¿Cómo integrarías SonarQube en un pipeline CI/CD de GitHub Actions? Describe los pasos a alto nivel.

La integración persigue que el análisis deje de ser una acción manual y voluntaria para convertirse en una verificación automática, ejecutada ante cada cambio y capaz de detener la entrega cuando el código no satisface el estándar acordado.

El primer punto a resolver es la **conectividad**. Los runners alojados por GitHub no pueden alcanzar una instancia que responde en localhost, por lo que existen dos alternativas: exponer el servidor en una URL accesible desde internet, protegida con TLS y restricciones de origen, o bien emplear un runner autoalojado dentro de la misma red donde reside el servidor. En un entorno corporativo la segunda opción suele resultar preferible, ya que evita exponer públicamente un servicio que concentra información detallada sobre las debilidades del código.

En segundo término se **registran los secretos del repositorio**. La URL del servidor y el token de análisis se configuran como secretos de la plataforma y nunca aparecen en el archivo de workflow, que sí se versiona junto con el código.

Los **disparadores del workflow** se definen sobre eventos de push a las ramas principales y sobre pull requests dirigidos a ellas. El análisis sobre pull requests es el de mayor valor práctico, ya que la retroalimentación llega antes de la fusión, momento en el cual el costo de corregir todavía es bajo y el autor del cambio conserva el contexto mental de lo que escribió.

La **preparación del entorno del job** requiere un checkout con profundidad completa, condición necesaria para que SonarQube pueda determinar la autoría y las fechas de cada línea y así distinguir el código nuevo del heredado. Un checkout superficial haría que la herramienta interpretara la totalidad del proyecto como código nuevo, invalidando el criterio de Clean as You Code. Se instalan asimismo el JDK requerido por el scanner y el SDK de .NET correspondiente.

Se habilita el **almacenamiento en caché** de los paquetes y de los binarios del scanner, con el fin de reducir el tiempo de ejecución del pipeline en corridas sucesivas, factor determinante para que el equipo no perciba el control como un obstáculo.

La **secuencia de análisis** replica los mismos pasos empleados de forma local: begin, compilación, pruebas con recolección de cobertura y end. Para pull requests se agregan las propiedades de decoración que publican los hallazgos como comentarios directamente sobre las líneas modificadas, integrando la retroalimentación en el flujo de revisión que el equipo ya utiliza.

El paso decisivo es la **verificación del Quality Gate y su vinculación con el resultado del job**. Se emplea una acción específica configurada para fallar cuando la puerta de calidad no se cumple. Sin este paso, el análisis se convierte en un reporte informativo que nadie consulta; con él, se transforma en un control efectivo. La **protección de la rama principal** completa el mecanismo, exigiendo que esa verificación figure entre los checks obligatorios para permitir la fusión; este es el punto donde el control adquiere carácter vinculante.

Finalmente, resulta conveniente **complementar con análisis programados**. Una ejecución periódica sobre la rama principal detecta vulnerabilidades recién publicadas en dependencias que ya formaban parte del proyecto, escenario que el análisis por cambios no cubre, dado que no hubo modificación alguna en el código.

Una consideración adicional atañe a los pull requests provenientes de bifurcaciones. Por diseño, GitHub no expone los secretos del repositorio a los workflows disparados desde forks, con el fin de impedir que un tercero los extraiga mediante un cambio malicioso en el propio pipeline. El patrón habitual para resolverlo consiste en dividir el flujo en dos, empleando un evento para la compilación y otro para el

análisis, de modo que este último se ejecute en el contexto de confianza del repositorio base.

5. Conclusiones

La práctica permite constatar que el análisis estático automatizado detecta, en cuestión de segundos, categorías completas de defectos cuya identificación mediante revisión manual resultaría costosa y poco confiable. Su mayor aporte no consiste en sustituir el criterio humano, sino en filtrar aquello que puede determinarse de forma mecánica, liberando el esfuerzo de revisión para los problemas de diseño y de lógica de negocio, donde el juicio profesional resulta insustituible.

La distinción entre bug, vulnerabilidad y security hotspot introduce una idea de fondo relevante para la materia: la seguridad no admite reducirse a una lista binaria de aprobado o reprobado. Existe una franja amplia de código sensible cuyo riesgo depende del contexto de uso, y esa franja exige revisión documentada, no automatización. El flujo de estados de los hotspots materializa exactamente ese principio dentro de la herramienta.

El manejo del token de análisis, por su parte, ofrece una lección coherente con el contenido del curso: los mismos principios que se aplican al código de la aplicación —privilegio mínimo, separación entre configuración y secretos, almacenamiento mediante hash— rigen también para las herramientas que se emplean para asegurarlo. Una cadena de suministro de software es tan segura como el eslabón más débil de su infraestructura de desarrollo.

Finalmente, la práctica evidencia que el valor real de la herramienta se concreta únicamente cuando se integra al pipeline y se le otorga capacidad de bloqueo. Un análisis ejecutado de forma manual y ocasional produce un tablero que se consulta cada vez menos; un análisis obligatorio que impide fusionar código incumpliendo el estándar modifica de manera efectiva el comportamiento del equipo. La diferencia entre ambos escenarios no es de naturaleza técnica sino de proceso.

6. Referencias

- SonarSource. *SonarQube Documentation* — Security Hotspots, Quality Gates, Clean as You Code.
- SonarSource. *SonarScanner for .NET* — Documentación de instalación y uso.
- OWASP. *OWASP Top 10:2021* — A03 Injection.
- OWASP. *Cheat Sheet Series* — SQL Injection Prevention Cheat Sheet.
- MITRE. *CWE-89: Improper Neutralization of Special Elements used in an SQL Command*.
- Microsoft Learn. *Entity Framework Core* — Raw SQL Queries y parametrización.
- Docker Inc. *Docker Compose specification*.
- GitHub Docs. *Using secrets in GitHub Actions y Removing sensitive data from a repository*.